

namaster-proof: Exact pseudo- C_ℓ window inference and tamper-evident provenance for reproducible spin-2 analyses

Houston Golden

July 16, 2026 — v2B.0.0

Abstract

namaster-proof is a focused Python verification layer for two error-prone steps in cut-sky spin-2 analyses. First, it evaluates a uniformly rotated EE, EB, BE, BB spectrum through the complete NaMaster bandpower-window operator, avoiding replacement of the operator by bin-centre or effective-multipole templates. Second, it publishes JSON results atomically with content-addressed sidecar receipts and fails closed when either the result or a declared execution contract changes. The package also supplies explicit multipole-support contracts, fixed-grid rotation-angle recovery, command-line receipt verification, and compatibility tests against the production helpers from which it was extracted. The software is intended for method validation and reproducibility checks; it is not a sky-analysis pipeline, foreground model, or cosmological inference engine.

1 Summary

Pseudo- C_ℓ estimators make masked spin-2 analyses computationally practical by relating full-sky spectra to coupled and decoupled bandpowers [1, 2]. Reproducible use of these estimators requires more than recording a best-fit number: the same window operator must be used in simulation and inference, harmonic limits must agree across fields and bins, and intermediate results must remain bound to their execution metadata.

namaster-proof packages these checks behind a small public API. Its numerical module constructs exact windowed responses for uniform polarization rotation, evaluates them at one or many angles, recovers an angle on a declared grid, and compares the window contraction with NaMaster’s couple–decouple operator. Its provenance module atomically publishes a JSON result and a SHA-256-bound receipt, then verifies the pair and optional metadata. A third module defines deterministic multipole and binning contracts. NumPy is the only required dependency; a NaMaster workspace is supplied by the user when exercising the physical window operator.

2 Statement of Need

For two spin-2 fields, NaMaster exposes a bandpower-window tensor with spectrum ordering $[EE, EB, BE, BB]$. A common shortcut is to evaluate a theory spectrum at a representative multipole for each bin. That shortcut is not generally identical to applying the complete mode-coupling and binning operator, and a mismatch between the simulation and fitting operators can appear as a parameter-recovery bias.

A separate reproducibility risk arises in long or sharded calculations. Ordinary filenames do not prove that a summary corresponds to the intended seeds, realization count, configuration, or

result bytes. A process can also terminate between writing a result and its metadata, leaving a plausible but incomplete artifact. These problems are usually addressed with analysis-local helpers. `namaster-proof` provides a reusable, tested implementation with explicit failure behavior, while remaining narrow enough to audit.

3 Software Architecture

The package is divided into three modules:

windows	Rotation of EE, EB, BE, BB spectra, exact contraction with a <code>get_bandpower_windows()</code> tensor, fixed-grid angle recovery, and an equivalence check against <code>couple_cell()</code> followed by <code>decouple_cell()</code> .
Multipoles	Validation of a common harmonic cutoff and construction of integer bandpower edges whose final exclusive edge is $\ell_{\max} + 1$.
receipts	Atomic JSON publication, streaming SHA-256 calculation, sidecar-receipt construction, and fail-closed content and metadata validation.

The functions accept array-like inputs and return NumPy arrays or plain Python dictionaries. NaMaster is deliberately not an installation dependency: window functions use the small workspace protocol described above, which makes the algebra testable with synthetic linear operators and permits users to pin PyMaster independently.

4 Exact-Window Inference

For an initially vanishing EB spectrum and a uniform rotation angle β , the rotated spectra can be decomposed into angle-independent, $\cos(4\beta)$, and $\sin(4\beta)$ components. In particular,

$$C_\ell^{EE}(\beta) = \frac{1}{2}(C_\ell^{EE} + C_\ell^{BB}) + \frac{1}{2}(C_\ell^{EE} - C_\ell^{BB}) \cos(4\beta), \quad (1)$$

$$C_\ell^{BB}(\beta) = \frac{1}{2}(C_\ell^{EE} + C_\ell^{BB}) - \frac{1}{2}(C_\ell^{EE} - C_\ell^{BB}) \cos(4\beta), \quad (2)$$

$$C_\ell^{EB}(\beta) = C_\ell^{BE}(\beta) = \frac{1}{2}(C_\ell^{EE} - C_\ell^{BB}) \sin(4\beta). \quad (3)$$

Let $W_{b\ell}^{ij}$ be the workspace tensor mapping input spectrum j to output spectrum i in band b . The package precontracts each of the three components,

$$R_b^{i,k} = \sum_{j,\ell} W_{b\ell}^{ij} C_\ell^{j,k}, \quad (4)$$

and subsequently evaluates

$$C_b^i(\beta) = R_b^{i,0} + \cos(4\beta) R_b^{i,c} + \sin(4\beta) R_b^{i,s}. \quad (5)$$

This retains the full window tensor while making repeated grid evaluation inexpensive. The recovery function minimizes a declared weighted or unweighted squared residual over a user-provided grid; its default is 2001 points spanning -1° to $+1^\circ$.

5 Deterministic Execution and Provenance

`publish_json` serializes a mapping with sorted keys, writes and flushes a process-specific temporary file, atomically replaces the destination, and synchronizes the containing directory. It then publishes a sidecar receipt containing schema version, result filename, byte count, SHA-256 digest, and caller-supplied execution metadata. The content-binding fields are protected from metadata override.

`verify_json_receipt` requires both files, parses both as JSON objects, and recomputes every protected field. `validate_json_receipt` additionally checks declared metadata such as a suite identifier, seed range, or realization count. Missing files, malformed objects, changed bytes, changed receipts, or contract mismatches raise an exception. The `namaster-proof verify` and `namaster-proof validate` commands expose the same behavior to shell workflows and return a nonzero status on failure.

6 Validation and Tests

Version 0.1.0 contains 19 automated tests covering numerical behavior, invalid inputs, the CLI, receipt mutation, protected fields, and compatibility with the original production helpers. The window suite uses an exactly represented synthetic linear workspace to compare the precontracted response with the couple-decouple route and to recover an injected grid angle. Rotation tests check conservation of total $EE + BB$ auto-power. Receipt tests mutate result bytes and receipt digests independently and require rejection.

Compatibility tests load the pre-package production modules and compare windowed bandpowers at negative, zero, and positive angles with zero requested absolute or relative tolerance. They also compare algebraic spectrum rotation, bandpower edges, and harmonic keyword contracts. In the associated physical production artifact, a workspace of shape $[4, 20, 4, 1025]$ gave a maximum absolute difference of 1.41×10^{-18} between direct window contraction and the couple-decouple operator. This number validates that recorded configuration; it is not a universal error bound.

7 Worked Examples

Minimal synthetic operator. The documentation constructs an identity-like workspace, builds a response from synthetic EE and BB arrays, evaluates a known rotation, and checks operator equivalence. This example has no dependency on a paper-specific dataset and is suitable for installation tests.

Synthetic CMB recovery campaign. The production use case employed CAMB 1.6.6 raw EE/BB spectra, PyMaster 2.6, $N_{\text{side}} = 512$, $\ell_{\text{max}} = 1024$, and deterministic seeds. For each of two nonzero injected angles, a 500-realization, foreground-free synthetic run recovered the injection from the mean bandpowers on the declared 0.001° grid: $0.270^\circ \mapsto 0.270^\circ$ and $0.342^\circ \mapsto 0.342^\circ$. The null injection recovered 0.000° . These are software-recovery checks under the stated simulation contract, not measurements, detection significances, or evidence for a physical birefringence model.

Sharded result validation. Production shards record fields such as suite, realization count, and seed range alongside the content digest. Aggregation can therefore reject a shard whose bytes were altered or whose declared range does not match the requested campaign, rather than silently combining it with valid results.

8 Limitations

`namaster-proof` does not create masks or maps, estimate covariance matrices, separate cosmic rotation from instrumental polarization calibration, model foregrounds, choose a likelihood, or infer cosmological parameters. Uniform rotation and initially vanishing EB are assumptions of the optimized three-component response; the general algebraic rotation helper accepts all four input spectra, but spatially varying rotation requires another model. Fixed-grid recovery inherits the range and resolution selected by the caller. Receipt validation establishes file integrity and agreement with declared metadata, not the scientific correctness of that metadata. SHA-256 sidecars are tamper-evident bindings, not digital signatures or an authorization system. Schema version 1 has no migration requirement yet; a future incompatible schema will require an explicit reader or conversion path.

9 Availability

Version 0.1.0 is distributed under the MIT License in the [BigBounce repository](#). Its source is the `namaster-proof` package directory. The package requires Python 3.10 or later and NumPy 1.24 or later. Installation, API, CLI, and development commands are documented in the package README; citation metadata are supplied in `CITATION.cff`. A persistent archival DOI and an independent package index release are not yet available and must not be inferred from the source repository. The complete synthetic-production artifacts used in the worked example are retained in the same repository’s `reproducibility` directory, under the `p1_namaster_500mc` subdirectory.

Acknowledgements

The exact-window operations build on NaMaster [2] and the MASTER pseudo- C_ℓ method [1]. The production example used CAMB [3] and HEALPix [4].

References

- [1] E. Hivon, K. M. Górski, C. B. Netterfield, B. P. Crill, S. Prunet, and F. Hansen, “MASTER of the CMB anisotropy power spectrum: A fast method for statistical analysis of large and complex cosmic microwave background data sets,” *Astrophysical Journal* 567, 2 (2002), [doi:10.1086/338126](#).
- [2] D. Alonso, J. Sanchez, and A. Slosar, “A unified pseudo- C_ℓ framework,” *Monthly Notices of the Royal Astronomical Society* 484, 4127–4151 (2019), [doi:10.1093/mnras/stz093](#).
- [3] A. Lewis, A. Challinor, and A. Lasenby, “Efficient computation of cosmic microwave background anisotropies in closed Friedmann–Robertson–Walker models,” *Astrophysical Journal* 538, 473–476 (2000), [doi:10.1086/309179](#).
- [4] K. M. Górski, E. Hivon, A. J. Banday, B. D. Wandelt, F. K. Hansen, M. Reinecke, and M. Bartelmann, “HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere,” *Astrophysical Journal* 622, 759–771 (2005), [doi:10.1086/427976](#).